From Microbial Ecosystem Resilience to Adaptive AI/HPC Architectures: A Framework for Safe and Verified Functional Recovery

Dr. Patrick Lemoine
Independent Researcher / HPC and AI Systems Engineer
Email: [lemoine_patrick@yahoo.ca] ORCID: [0009-0008-9465-8159]

***Abstract*—Artificial Intelligence and High-Performance Computing systems increasingly operate as heterogeneous, distributed, and resource-constrained infrastructures. Conventional fault-tolerance mechanisms such as checkpoint/restart, replication, and process restart remain essential, but they primarily focus on restoring execution after detectable infrastructure failures. Structural recovery—restarting processes or restoring execution—does not necessarily ensure functional recovery: the restoration of essential capabilities with verified state integrity, output correctness, and safety. Nor does it ensure that critical functions remain available during disruption or that degraded outputs are acceptable. This paper introduces BioRes-AI/HPC, a bio-inspired resilience framework that translates principles observed in microbial ecosystems into concrete architectural mechanisms for AI and HPC workflows. The framework maps functional redundancy, modularity, local sensing, adaptive reconfiguration, and functional continuity to selective replication, failure-domain isolation, multi-layer observability, topology-aware scheduling, graceful degradation, and verified recovery. BioRes-AI/HPC distinguishes three interacting degradation domains: infrastructure failures, data perturbations, and AI decision degradation. It defines safe and verified recovery as functional restoration combined with post-recovery validation of integrity, correctness, quality, and safety constraints. The paper further proposes a functional-dependency model, a resilience controller, and an evaluation methodology based on detection time, blast radius, functional continuity, safe recovery time, correctness after recovery, and resilience overhead. The proposed framework provides a systematic path toward AI/HPC infrastructures that remain useful, predictable, and verifiable under partial failures and uncertain operating conditions.**

***Index Terms*—Artificial intelligence, high-performance computing, resilience engineering, fault tolerance, distributed systems, bio-inspired computing, adaptive scheduling, functional continuity.**

I. Introduction

Artificial Intelligence (AI) and High-Performance Computing (HPC) are converging into increasingly complex computational infrastructures. Contemporary scientific and industrial workflows combine heterogeneous compute nodes, GPUs, high-speed interconnects, distributed storage, simulation codes, machine-learning models, workflow engines, schedulers, data pipelines, and external software services.

These systems are typically optimized for nominal operating conditions. Performance evaluation commonly focuses on throughput, floating-point operations per second, training throughput, inference latency, resource utilization, energy efficiency, or job completion time. These metrics remain essential, but they provide only a partial view of system quality.

Production AI/HPC systems do not operate continuously under ideal assumptions. A distributed AI training job can experience loss of a GPU, a node, a network path, or an I/O service. An HPC simulation can be delayed or corrupted by storage degradation, transient hardware errors, scheduler failures, or communication faults. An AI pipeline can remain technically available yet functionally compromised while receiving incomplete, delayed, corrupted, shifted, or out-of-distribution data. A model can produce low-confidence or unsafe decisions while the infrastructure appears healthy.

These situations reveal an important limitation of conventional fault-tolerance approaches. Restarting a job or restoring a checkpoint may re-establish execution, but it does not automatically ensure that the recovered state is correct, that the input data remain valid, or that the resulting AI decision is trustworthy.

This paper argues that resilience for AI/HPC systems should be defined not only as the ability to survive, restart, or recover after a failure, but also as the ability to preserve essential functions, operate safely in degraded modes, and verify recovery outcomes.

Microbial ecosystems offer a useful conceptual model. Microbial communities face environmental disturbances, resource fluctuations, species loss, pollution, and competition. Their resilience does not require that the system remain structurally unchanged. Rather, resilience may involve reorganization while preserving or recovering critical ecosystem functions [1], [2], [3].

The goal of this paper is not to claim that biological mechanisms can be copied directly into computing infrastructures. Instead, it translates selected resilience principles from microbial ecology into explicit architectural abstractions, implementation mechanisms, and evaluation metrics for AI/HPC systems.

The main contributions of this paper are as follows:

1) A biological-to-computational translation of microbial resilience concepts into AI/HPC architectural principles.
2) The BioRes-AI/HPC framework, which combines multi-layer observability, functional dependency modeling, adaptive resilience control, and verified recovery.

3) A distinction among infrastructure failures, data perturbations, and AI decision degradation.
4) A definition of safe and verified recovery that extends conventional recovery-time metrics.
5) A resilience evaluation methodology for heterogeneous AI/HPC workflows under controlled fault-injection scenarios.

II. Background and Related Work

A. Microbial Community Resilience

In ecological systems, resistance and resilience are related but distinct concepts. Resistance refers to the ability of a community to remain relatively unchanged under perturbation. Resilience commonly refers to the capacity or rate of recovery after disturbance [1], [2].

Allison and Martiny examined resistance, resilience, and redundancy in microbial communities, highlighting the role of functional redundancy in maintaining ecosystem processes despite changes in community composition [1]. Shade *et al.* further identified multiple biological attributes that influence the stability of microbial communities, including diversity, population-level adaptation, dispersal, and interaction structure [2]. Philippot *et al.* reviewed microbial community resilience across ecosystems and multiple disturbances, emphasizing that microbial responses depend on the type, intensity, duration, recurrence, and interaction of perturbations [3].

For computing systems, the relevant abstraction is not microbial species or microbial evolution. Instead, the applicable insight is that a system may preserve critical functions despite the failure, replacement, degradation, or reorganization of some components.

B. Resilience in HPC Systems

HPC resilience has traditionally focused on protecting applications from faults, errors, and failures at large scale. Common approaches include checkpoint/restart, rollback recovery, replication, algorithm-based fault tolerance, failure prediction, error detection, and runtime adaptation.

Hukerikar and Engelmann introduced resilience design patterns as reusable building blocks for systematic design, evaluation, and optimization of resilience mechanisms in extreme-scale HPC systems [4]. Their framework highlights the need to coordinate resilience mechanisms across hardware, operating systems, runtime environments, middleware, and applications.

However, conventional HPC resilience is often oriented toward restoring execution after a component-level or process-level fault. This approach is necessary, but increasingly insufficient for AI/HPC workflows in which data validity, model confidence, decision quality, and external dependencies can fail independently of the underlying hardware.

C. Silent Data Corruption and Correctness

Silent data corruption is particularly challenging because it may not produce a process crash, hardware alert, or obvious runtime failure. Instead, corruption can propagate through memory, communication buffers, numerical kernels, model parameters, checkpoints, or output artifacts.

Berrocal *et al.* proposed runtime data-analysis methods to detect silent data corruptions in HPC applications, demonstrating that crash-oriented fault management alone is insufficient when applications can continue executing with corrupted state [5].

Therefore, a restart is not equivalent to a correct recovery. A resilient system must determine whether the restored state, intermediate artifacts, and final outputs satisfy predefined correctness constraints. This distinction motivates the problem statement in Section III: infrastructure availability must be assessed alongside data validity and AI decision quality, rather than treated as evidence of functional correctness.

D. AI System Reliability and Data-Driven Degradation

Machine-learning systems introduce additional failure modes beyond infrastructure faults. Their operation depends on data pipelines, feature definitions, model versions, external services, configuration states, feedback loops, and assumptions about the operating environment.

Sculley *et al.* identified hidden technical debt in machine-learning systems, showing that the complexity of operational ML systems often arises from dependencies and interactions that extend well beyond the trained model itself [6].

The NIST AI Risk Management Framework identifies validity, reliability, safety, security, and resilience as essential characteristics of trustworthy AI systems [7]. These characteristics imply that a technically available model cannot automatically be considered reliable or safe when its input regime, operating environment, or output constraints have changed.

III. Problem Statement

A. Limitation of Execution-Centric Recovery

Traditional recovery mechanisms generally answer the following question: can execution resume after a detected failure? For AI/HPC systems, this question is incomplete. A more appropriate question is: can the system restore an essential function while ensuring that its recovered state, inputs, and outputs remain valid, safe, and trustworthy?

An HPC application may restart from a checkpoint that already contains corrupted numerical state. A distributed training job may recover after losing a worker but continue with incomplete or inconsistent data. An inference service may remain online while processing inputs outside the distribution under which its model was validated.

Consequently, operational recovery does not necessarily imply functional recovery, and functional recovery does not necessarily imply safe recovery.

B. Three Degradation Domains

BioRes-AI/HPC distinguishes three interacting degradation domains.

1) Infrastructure Failures: Infrastructure failures affect the computational substrate. Examples include GPU failures, node loss, memory exhaustion, storage degradation, network partitions, communication-runtime failures, scheduler failures, and unavailable dependencies. Hardware telemetry, scheduler states, operating-system events, CUDA/NCCL/MPI errors, storage logs, and runtime exceptions can reveal these events. However, component-level monitoring alone may miss their effects on dependent functions or fail to establish whether a resumed workflow is semantically correct.

2) Data Perturbations: Data perturbations affect the validity, completeness, timeliness, consistency, or statistical representativeness of data. Examples include missing values, corrupted records, schema changes, delayed streams, noisy sensor data, data drift, and out-of-distribution inputs. Unlike infrastructure failures, these perturbations may not stop execution. Their primary danger is semantic: the workflow may continue running while processing information that violates model assumptions or scientific constraints. Conventional availability and resource monitoring can therefore report nominal operation while data-aware validation would reject the inputs.

3) AI Decision Degradation: AI decision degradation concerns the trustworthiness of the system output. It may include low confidence, excessive uncertainty, poor calibration, unstable predictions, constraint violations, unexpected output distributions, or degraded performance on a critical task. This domain concerns output-level acceptance criteria rather than component availability or input validity alone, although the three domains can interact. Conventional infrastructure monitoring and basic data-quality checks may miss this degradation: a model can generate an unreliable decision even when both report nominal operation.

C. Safe and Verified Recovery

This paper defines *safe and verified recovery* as the restoration of an essential system function after disruption, accompanied by verification that the recovered system state and resulting outputs satisfy predefined integrity, correctness, quality, and safety constraints.

$$\begin{aligned}\text{Safe and Verified Recovery} = \text{Functional Restoration} \\ \wedge\ \text{Post-Recovery Verification}.\end{aligned} \tag{1}$$

Functional restoration concerns whether the essential service or computational workflow is operational again, possibly in a validated degraded mode. Post-recovery verification concerns whether the recovered state and outputs satisfy the declared acceptance constraints. Resumed execution alone satisfies neither condition by default.

For a sequential recovery procedure, the safe recovery time is defined as:

$$\begin{aligned}T_{\mathrm{SR}} = T_D + T_I + T_{\mathrm{RC}} \\ + T_{\mathrm{FR}} + T_V.\end{aligned} \tag{2}$$

Here, T_D is the detection time, T_I is the isolation and containment time, T_{RC} is the reconfiguration time, T_{FR} is the restoration time, and T_V is the post-recovery verification time.

A system should not be considered fully recovered until verification has completed successfully. The additive expression assumes non-overlapping phases; when phases overlap or retries occur, T_{SR} is measured as elapsed time from disturbance onset to successful verification. Section VI specifies the corresponding evaluation metrics.

IV. BioRes-AI/HPC Architecture

A. Architectural Overview

BioRes-AI/HPC is composed of four interacting layers: (i) a sensing and observability layer, (ii) a functional dependency layer, (iii) an adaptive resilience controller, and (iv) a verified recovery layer. The architecture is designed to support AI inference services, distributed training workloads, simulation workflows, coupled AI/HPC applications, and heterogeneous GPU clusters.

B. Sensing and Observability Layer

The sensing layer collects signals from infrastructure, runtimes, data pipelines, and model-serving components. Infrastructure signals include node availability, GPU health and error counters, CPU and memory utilization, storage latency, filesystem availability, network congestion, scheduler status, CUDA/NCCL/MPI exceptions, and checkpoint status.

Data signals include missing-value rate, schema conformance, input completeness, data latency, feature-range violations, statistical shift indicators, and out-of-distribution scores. Decision signals include model confidence, predictive uncertainty, calibration status, output consistency, and domain-specific safety or scientific constraints.

C. Functional Dependency Model

The framework models not only physical resources, but also the functions that must be preserved. For a generic AI/HPC workflow, the function set is represented as:

$$\begin{aligned}\mathcal{F} = \{f_{\mathrm{data}}, f_{\mathrm{train}}, f_{\mathrm{ckpt}}, \\ f_{\mathrm{inf}}, f_{\mathrm{val}}, f_{\mathrm{orch}}\}.\end{aligned} \tag{3}$$

where f_{data}, f_{train}, f_{ckpt}, f_{inf}, f_{val}, and f_{orch} denote data ingestion and preparation, training, checkpointing, inference, validation, and workflow orchestration, respectively.

Each critical function f_i is represented by:

$$f_i = (c_i, D_i, A_i, Q_i, R_i), \tag{4}$$

where c_i is the criticality level, D_i is the set of functional and resource dependencies, A_i is the set of validated alternatives or fallback mechanisms, Q_i is the set of quality, correctness, and safety constraints, and R_i is the recovery policy.

This representation allows the system to distinguish between a failed component and a failed capability. For example, losing one GPU does not necessarily mean that distributed training has failed. The workflow may continue with reduced parallelism, reconfigured process groups, or an alternative

resource allocation. By contrast, loss of checkpoint integrity may make recovery unsafe even if compute resources remain available.

D. Adaptive Resilience Controller

The resilience controller receives state information from the observability layer and applies an application-aware policy:

$$\pi : (S_{\text{infra}}, S_{\text{data}}, S_{\text{decision}}, C_{\text{workflow}}) \rightarrow A_{\text{resilience}}, \quad (5)$$

where S_{infra} represents infrastructure state; S_{data} represents data validity and quality state; S_{decision} represents AI output quality and confidence state; C_{workflow} represents workflow criticality, constraints, and objectives; and $A_{\text{resilience}}$ represents a selected resilience action.

Potential actions include isolating an unhealthy node or service, restarting from a verified checkpoint, migrating a workload to healthy resources, reconfiguring distributed parallelism, reducing batch size or model precision, switching to a validated fallback model, delaying non-critical tasks, quarantining invalid data, rejecting or abstaining from a low-confidence AI decision, and escalating to human review.

E. Verified Recovery Layer

The verified recovery layer evaluates whether a recovery action restored the required function and whether the resulting state is trustworthy. Verification may include checkpoint checksums and metadata validation, consistency of distributed process states, numerical residual checks, conservation-law checks for scientific simulations, model artifact validation, input-data schema validation, confidence thresholds, output constraints, and domain-specific safety conditions.

The recovery process transitions through the following states:

$$\begin{aligned} \text{Nominal} \rightarrow \text{Detected} \rightarrow \text{Isolated} \\ \rightarrow \text{Degraded} \rightarrow \text{Restored} \rightarrow \text{Verified}. \end{aligned} \quad (6)$$

The system should transition to the *Verified* state only when relevant integrity, correctness, and safety criteria have been satisfied.

V. Bio-Inspired Resilience Patterns

A. Functional Redundancy

Computing equivalent: Prevalidated alternative functional paths.

Microbial communities can retain ecosystem functions despite variation in species composition, partly because multiple organisms may contribute to similar functions [1]. In AI/HPC systems, functional redundancy should not be interpreted as indiscriminate replication of every component. Instead, it refers to prevalidated alternative paths for delivering essential functions.

Examples include redundant metadata or checkpoint services, multiple execution backends, replica services for critical orchestration functions, reduced but validated fallback models, alternative numerical kernels, alternate storage paths, redundant communication routes, and resource pools reserved for high-criticality jobs.

B. Modularity and Failure Containment

Computing equivalent: Failure-domain isolation and dependency-aware containment.

Ecological systems can limit the propagation of local disturbances through spatial structure and interaction boundaries. In computing systems, modularity can reduce blast radius. Relevant mechanisms include failure domains, service isolation, namespace and resource partitioning, job-level containment, data-pipeline quarantine, network segmentation, independent control-plane components, and per-workflow policy boundaries.

C. Local Sensing and Early Detection

Computing equivalent: Source-proximal, multi-layer observability.

Microbial systems respond to local chemical and environmental signals. Analogously, AI/HPC systems should detect anomalies as close as possible to their source. Examples include node-level telemetry agents, GPU health monitoring, runtime-level error detectors, input validation at ingestion boundaries, out-of-distribution detection before inference, per-model confidence monitoring, and local checkpoint validation.

D. Adaptive Reconfiguration

Computing equivalent: Policy-controlled resource and workflow reconfiguration.

A resilient ecosystem can reorganize after disturbance. AI/HPC systems can implement controlled reconfiguration through fault-aware scheduling, topology-aware resource allocation, dynamic process-group resizing, workload migration, replica activation, resource reallocation, adaptive precision, fallback model selection, and priority-aware job management.

E. Graceful Functional Degradation

Computing equivalent: Validated degraded service with explicit acceptance constraints.

Graceful degradation preserves essential service when nominal performance cannot be maintained. For AI systems, this may involve reduced inference scope, fallback to a smaller validated model, deferred processing, human review, or abstention. For HPC systems, it may involve reduced parallelism, prioritization of critical simulations, suspension of low-priority workloads, reduced output frequency, or continuation from a validated checkpoint with fewer resources.

The goal is not to maximize output at any cost. The goal is to sustain acceptable and trustworthy function under constrained conditions.

VI. Resilience Metrics

The following metrics operationalize the degradation domains and recovery definition introduced in Section III. They evaluate preserved function and verified outcomes, rather than execution availability alone.

A. Detection Time

The detection time is:

$$T_D = t_{\text{detected}} - t_{\text{fault}}, \tag{7}$$

where t_{fault} is the beginning of the perturbation and t_{detected} is the time at which the system identifies the event.

B. Blast Radius

The blast radius quantifies the propagation of a disturbance:

$$B_R = \frac{|\mathcal{F}_{\text{affected}}|}{|\mathcal{F}_{\text{total}}|}, \tag{8}$$

where $\mathcal{F}_{\text{affected}}$ is the set of functions affected by the incident and $\mathcal{F}_{\text{total}}$ is the total set of functions in the workflow. A lower B_R indicates better containment.

C. Functional Continuity

Functional continuity quantifies the useful service maintained during a disturbance:

$$FC = \frac{1}{T}\int_0^T U(t)\,dt, \tag{9}$$

where $U(t) \in [0,1]$ is the normalized system utility at time t. Nominal operation corresponds to $U(t) = 1$, verified degraded operation corresponds to $0 < U(t) < 1$, and unavailable or invalid service corresponds to $U(t) = 0$.

For an AI service, $U(t)$ may include response availability, confidence, quality, latency, and safety compliance. For an HPC workflow, it may include execution progress, numerical correctness, checkpoint validity, and resource efficiency.

D. Safe Recovery Time

Using the phase notation in (2), safe recovery time can equivalently be written as:

$$T_{\text{SR}} = T_D + T_I + T_R + T_V, \tag{10}$$

where $T_R = T_{\text{RC}} + T_{\text{FR}}$ groups reconfiguration and functional restoration; T_D, T_I, and T_V retain their definitions from Section III. For sequential phases, this expression is identical to (2). In measurements, T_{SR} ends only after the required state and output checks succeed. If verification does not succeed within the observation window, recovery is reported as incomplete rather than assigned a successful recovery time.

E. Correctness After Recovery

Correctness after recovery is represented as:

$$C_R = \begin{cases} 1, & \text{if all required verification constraints are satisfied,} \\ 0, & \text{otherwise.} \end{cases} \tag{11}$$

For applications requiring a continuous measure, C_R can be represented as a weighted quality score:

$$C_R = \sum_{j=1}^{n} w_j v_j, \tag{12}$$

where v_j is the verification result for constraint j, and w_j is its importance weight.

F. Resilience Overhead

Resilience has a cost. The framework must evaluate overhead in runtime, energy, memory, storage, and reserved capacity:

$$O_R = \alpha O_{\text{time}} + \beta O_{\text{energy}} + \gamma O_{\text{memory}} + \delta O_{\text{storage}}. \tag{13}$$

The parameters α, β, γ, and δ can be selected according to application priorities.

VII. Experimental Methodology

A. Testbed

The proposed evaluation should use a heterogeneous GPU cluster managed by Slurm or an equivalent resource manager. The platform should include multiple GPU-equipped compute nodes, high-speed interconnects, shared or distributed storage, distributed AI frameworks such as PyTorch Distributed, NCCL or MPI communication layers, workflow or API orchestration, infrastructure telemetry collection, and fault-injection tooling.

B. Workloads

The evaluation should include at least three workload categories:

1) Distributed AI training using data-parallel or model-parallel execution.
2) AI inference service with confidence monitoring and validated fallback modes.
3) Coupled AI/HPC workflow combining simulation, data processing, AI inference, and checkpointing.

The inclusion of coupled workflows is important because it exposes interactions among runtime faults, data validity, and decision quality.

C. Fault-Injection Scenarios

The test plan should include controlled and repeatable perturbations. Infrastructure scenarios include GPU reset or unavailability, process termination, node failure, network slowdown or partition, NCCL communication error, storage latency increase, checkpoint-write failure, and memory-pressure events.

Data scenarios include missing input partitions, corrupted data records, schema mismatches, delayed input streams, feature-range violations, distribution shifts, and out-of-distribution inputs.

AI decision scenarios include low-confidence outputs, uncertainty-threshold violations, calibration failures, output-constraint violations, model-version mismatches, and unexpected prediction distributions.

D. Baselines

E. Research Hypotheses

The following hypotheses can be evaluated:

- **H1:** BioRes-AI/HPC improves functional continuity compared with abort-and-restart and conventional checkpoint/restart approaches.

TABLE I
BASELINE STRATEGIES FOR EXPERIMENTAL EVALUATION

Baseline	Description
B1: Abort and restart	The workflow aborts after a detected fault and restarts manually or automatically.
B2: Conventional checkpoint/restart	The workflow restores the most recent checkpoint after an infrastructure failure.
B3: Fault-aware recovery	The workflow performs detection and rescheduling but does not model data validity or AI decision degradation.
B4: BioRes-AI/HPC	The proposed framework with multi-layer detection, functional dependency modeling, graceful degradation, and verified recovery.

- **H2:** Local observability and failure-domain isolation reduce blast radius and detection time.
- **H3:** Verified recovery reduces the probability of accepting corrupted or invalid post-recovery outputs.
- **H4:** Selective functional redundancy provides a better resilience-to-overhead trade-off than uniform replication.
- **H5:** Adaptive fallback and controlled degradation preserve useful service under resource constraints better than complete service interruption.

VIII. DISCUSSION

A. Why Biological Inspiration Is Useful

The biological analogy is useful because it changes the design objective. Instead of designing only for maximum efficiency in nominal conditions, it encourages the design of systems capable of preserving critical capabilities during uncertainty.

Microbial ecosystems demonstrate that stability does not necessarily require structural immutability. A system can change internally while maintaining or recovering essential functions. For AI/HPC systems, this supports an architectural shift from component-centric recovery toward function-centric continuity.

B. Limits of the Analogy

The analogy must not be overstated. Microbial communities evolve over long time scales, while engineered systems must react within seconds, minutes, or hours. Biological redundancy, competition, cooperation, and adaptation do not map directly to hardware replication, scheduler policies, or model fallbacks.

The purpose of BioRes-AI/HPC is therefore not literal biomimicry. It is a disciplined translation of selected resilience principles into measurable system-design mechanisms.

C. Trade-Offs

Resilience mechanisms introduce overhead. Checkpointing consumes storage and I/O bandwidth. Replication consumes compute capacity. Monitoring produces telemetry overhead. Validation can increase recovery time. Fallback models may reduce accuracy or functionality. Human review increases latency and operational cost.

Therefore, the design problem is multi-objective:

$$\begin{aligned} \max \quad & (FC, C_R, \text{efficiency}) \\ \text{subject to} \quad & T_{\mathrm{SR}} \le T_{\mathrm{SR}}^{\max}, \\ & O_R \le O_R^{\max}, \\ & \text{safety constraints are satisfied.} \end{aligned} \tag{14}$$

The appropriate balance depends on workload criticality, acceptable risk, energy budget, hardware availability, and application-specific correctness requirements.

D. Limitations

This paper proposes a framework and evaluation methodology. It does not claim that all resilience patterns are universally beneficial or that one set of metrics is sufficient for every AI/HPC workload. Quality and safety constraints are domain-specific; out-of-distribution and uncertainty detection remain imperfect; fault injection may not reproduce all real-world failures; multi-layer monitoring can create non-negligible overhead; and recovery verification must be adapted to the numerical, scientific, or operational semantics of each workload.

IX. CONCLUSION

This paper introduced BioRes-AI/HPC, a bio-inspired framework for resilient AI and HPC architectures. The proposed approach translates microbial resilience concepts into five engineering principles: functional redundancy, modularity, local sensing, adaptive reconfiguration, and functional continuity. It extends conventional fault tolerance by distinguishing infrastructure failures, data perturbations, and AI decision degradation.

The central contribution is the concept of *safe and verified recovery*. Structural recovery restores processes, resources, or execution; functional recovery restores an essential capability. Neither resumed execution nor nominal availability establishes semantic correctness. Recovery is accepted only when the required function has been restored and post-recovery checks confirm that the resulting state and outputs satisfy predefined integrity, correctness, quality, and safety constraints. This criterion supports trustworthy continuity without equating a successful restart with a verified outcome.

The framework defines a functional dependency model, a multi-layer resilience controller, and metrics for detection time, blast radius, functional continuity, safe recovery time, correctness after recovery, and resilience overhead.

Future work will implement BioRes-AI/HPC on a heterogeneous GPU cluster and evaluate its behavior under controlled

infrastructure, data, and AI decision degradations. The broader objective is to move AI/HPC resilience from an after-the-fact recovery mechanism toward a first-class architectural property.

REFERENCES

[1] S. D. Allison and J. B. H. Martiny, "Resistance, resilience, and redundancy in microbial communities," *Proceedings of the National Academy of Sciences*, vol. 105, suppl. 1, pp. 11512–11519, 2008, doi: 10.1073/pnas.0801925105.

[2] A. Shade *et al.*, "Fundamentals of microbial community resistance and resilience," *Frontiers in Microbiology*, vol. 3, Art. no. 417, 2012, doi: 10.3389/fmicb.2012.00417.

[3] L. Philippot, B. S. Griffiths, and S. Langenheder, "Microbial community resilience across ecosystems and multiple disturbances," *Microbiology and Molecular Biology Reviews*, vol. 85, no. 2, Art. no. e00026-20, 2021, doi: 10.1128/MMBR.00026-20.

[4] S. Hukerikar and C. Engelmann, "Resilience design patterns: A structured approach to resilience at extreme scale," *Journal of Supercomputing Frontiers and Innovations*, vol. 4, no. 3, pp. 4–42, 2017, doi: 10.14529/jsfi170301.

[5] E. Berrocal, L. Bautista-Gomez, S. Di, and Z. Lan, "Lightweight silent data corruption detection based on runtime data analysis for HPC applications," in *Proceedings of the 24th International Symposium on High-Performance Parallel and Distributed Computing*, 2015, pp. 275–278, doi: 10.1145/2749246.2749253.

[6] D. Sculley *et al.*, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems*, vol. 28, 2015.

[7] National Institute of Standards and Technology, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, Jan. 2023, doi: 10.6028/NIST.AI.100-1.